Higher Diploma in Computer Science

Computer Systems & Networks

SPRINGBOARD
www.springboardcourses.ie

Ireland's European Structural and Investment Funds Programmes 2014-2020
Co-funded by the Irish Government and the European Union

OIDEACHAS AGUS EOLAÍOCHTA | EDUCATION AND SCIENCE

HEA | HIGHER EDUCATION AUTHORITY
AN tÚDARÁS um ARD-OIDEACHAS

European Union
Investing in your future
European Social Fund

SETU
Ollscoil Teicneolaíochta an Oirdheiscirt
South East Technological University

Arithmetic Operators

- use “$((...))” to do math calculations

```
~$ echo "$(( 3 + (4+5) ))"
```

- TIP
  - escape (\) the asterisk (*) when **multiplying** to prevent pattern expansion in Bash [or any other bash wildcards]

```
compsys@compsys-virtualbox:~$ myvar=100
compsys@compsys-virtualbox:~$ expr ${myvar} \* 3
300
```

Arithmetic Operators - Overview

```
#!/bin/bash

x=10
y=3

echo $(( x + y ))   # addition
echo $(( $x + $y )) # also valid

echo $(( x - y ))   # subtraction
echo $(( $x - $y )) # also valid

echo $(( x * y ))   # multiplication
echo $(( $x * $y )) # also valid

echo $(( x / y ))   # division
echo $(( $x / $y )) # also valid

echo $(( x ** y ))  # exponentiation
echo $(( $x ** $y ))    # also valid
```

```
echo $(( x % y ))   # modular division
echo $(( $x % $y )) # also valid

(( x += 4 ))    # increment variable by constant
echo $x

(( x -= 4 ))    # decrement variable by constant
echo $x

(( x *= 4 ))    # mulitply variable by constant
echo $x

(( x /= 4 ))    # divide variable by constant
echo $x

(( x %= 4 ))    # Remainder of Dividing Variable by Constant
echo $x
```

Pipes

|

Working with Pipelines

- One of the most powerful Linux commands
- Syntax: |
- takes **output** from one command and **uses it as input** for another

√ Can link/chain multiple pipes

Main purpose: filtering

- The most popular use for pipes involves the commands **sort** and **grep**

DON'T CONFUSE pipe (**|**) redirection with file redirection (**>**) and (**<**).

cmd1 > file	→File redirection either sends output to a file
or	→instead of STDOUT (screen) ***OR***
cmd1 < file	→pulls input from file instead of STDIN →(keyboard)

pipe and sort

- **sort file on the basis of 1st three fields**:

```
$ sort -t"," -k1,3 employee.txt
```

Where

-t = option is used to provide the delimiter, assuming it's a comma

-k = specify the keys on the basis of which the sorting has to be done.

format of '-k' is : '-km,n' (*m* = the starting key and *n* = the ending key)

-f would sort and ignore case

-n would sort numerically

-c would sort and remove duplicates

grep pipe and sprt

- **We want to return** only the lines that do not contain the characters ‘Manager’, but the result should be in reverse order.

```
$ cat employee.txt | grep -vi Manager | sort - r
```

Counting files

- For example, you can list the number of files in the current directory:

$ls #sends output to STDOUT (screen) by default

$ls –l | less # pipe to less to make output easier to read/scrollable

$ ls | head -3 | tail -1 #you can stack(/chain) commands using pipes

$ls | head -3 | tail -1 > myoutput #you can combine with redirection

Count lines

$wc -l employee.txt #How many lines are there in your ***employee.txt*** file

- Using the **pipe** command, here's how to pipe the output of **ls** command into **wc**:

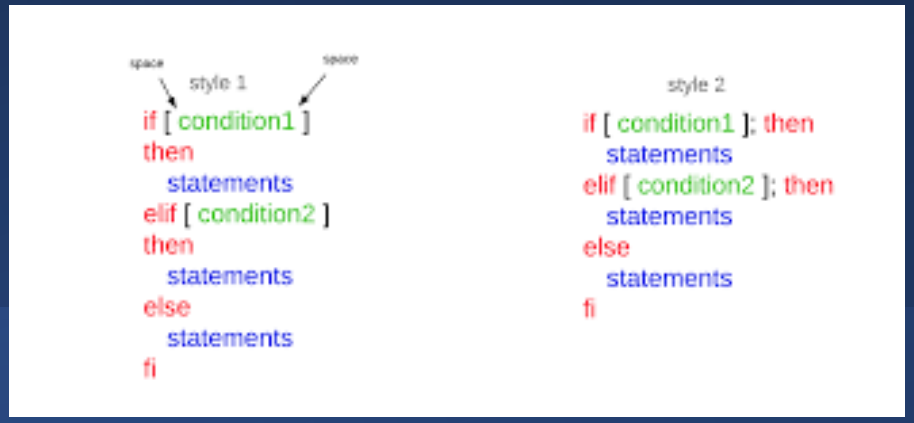
(i) $ls ~ | wc –l #how many files are ***home***

(ii) Pipe this to `less` to inspect, **q** to quit

(iii) $ _____ #how many lines are in the ***/etc***

Syntax	| *vertical line*
View a history of your commands, in scrollable format by piping the command to `less`:	$history | less
Try out	$history | head -5
Count the number of lines in the home directory by piping the ****STDOUT**** of the `ls` command into the ****STDIN**** of `wc`	$ls ~ | wc –l

Conditional Statements



test

test command – is true/false

```
Caroline@ICTSkills>~$ test 10 -gt 55; echo $?
1
```

Tests can be combined with logical AND and OR

```
compsys@compsys-virtualbox:~$ test 56 -gt 55 && echo true || echo false
true
compsys@compsys-virtualbox:~$ test 6 -gt 55 && echo true || echo false
false
```

Re-written using square brackets

```
compsys@compsys-virtualbox:~$ [ 56 -gt 55 ] && echo true || echo false
```

Arithmetic Operators

+ -	Addition, subtraction
++ --	Increment, decrement
* / %	Multiplication, division, remainder
**	Exponentiation

-eq	→	_____
-ge	→	_____
-gt	→	_____
-le	→	_____
-lt	→	_____
-ne	→	_____

If BASH double parenthesis i.e. (( )) are not used, then the **test** command must be used to compare integer variables


if

if statement

= does a string comparison and

–eq does a numerical comparison

if [***test*** **]**
then
commands
fi

```
if [ "$userinput" != "$password" ];
then
echo "Oops; Wrong Password Entered!"
exit 1
fi
```

How to use exit codes in scripts with an *if*

- Return a message to the user if this file called “employee.txt” ran successfully

What will be the output?

```
#!/bin/bash

cat employee.txt

if [ $? -eq 0 ]
then
        echo "This file $0 ran OK"
        exit 0
else
        echo "File failed"
        exit 1
fi
```

if/then/else

- Sometimes we may have a series of conditions that may lead to different paths.

if [***test*** **]**
then ***commands***

else ***commands***
fi

```
#!/bin/bash
if [ -f employee.bak ]
        then echo employee.bak regular
file exists!
        else echo employee.bak not found!
fi
```

How to use exit codes in scripts with an *if/then/else*

$? = the **exit status** of the last command executed

```
if [ $? -eq 0 ]; then
        # do something as the last command executed OK
else
        # do something else as last command executed with error
fi
exit
```

if then elif

- You can nest a new **if** inside an **else** with **elif**

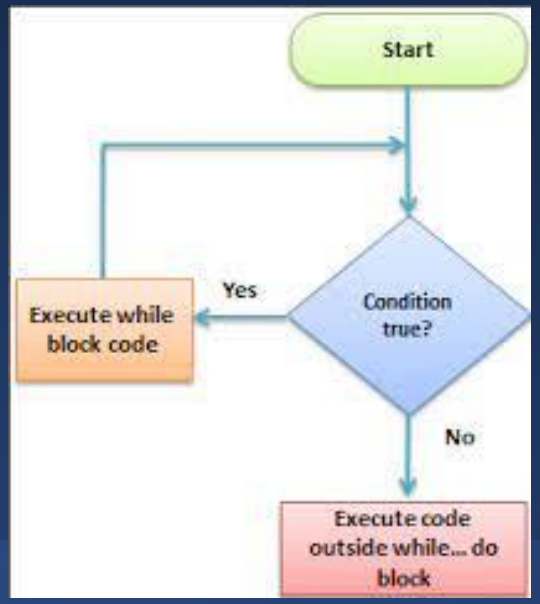
```
#!/bin/bash
count=42
if [ $count -eq 42 ]
then
        echo "42 is correct."
elif [ $count -gt 42 ]
        then
                echo "Too much."
        else
                echo “More!"
fi
```

$# Shell Variable

- is the number of arguments taken in

$echo $# #outputs the number of positional parameters of your script.

```
if [ $# -eq 1 ]
then
   echo $1
else
   echo "invalid argument please pass only one argument "
fi
exit
```



Loop Constructs

for

for

```
#!/bin/bash
# Basic range in loop
for value in {1..5}
do
        echo $value
done
echo All done!
```

General format:

```
for var in varlist
do
      commands
done
```

The **for** statement executes the commands once for each value in the list

<u>for example 1</u>

```
# prints the numbers from 101 to 105
#
for num in 1 2 3 4 5
do
    expr $num + 100
done
```

while

- general syntax for using the Bash while loop is

```
while [condition]
do
        //code to execute here
        //increment/decrement
 done
```

while

```
file=filename

while read -r line;
do
        echo $line
done < "$file"
```

Programming languages have loop constructs to allow a block of code to be executed repeatedly, either a fixed number of times, or while some condition holds.

`while` instruction general format

```
while [ condition ]
do
      commands
done
```

Example: The following script prints all numbers from 1 to 100

```
# Prints all numbers from 1 to 100
#
num=1
while [ $num -le 100 ]
do
  echo $num
  num=`expr $num + 1`
done
```

Infinite loops

CTRL+C

- the `true` indicates the while loop will **forever** iterate

If it returns quickly, it's beneficial to insert a small pause between executions to avoid CPU load, eg

```
while true
do
        echo "I will continue to iterate unless you press Ctrl+C“
        echo “let me also rest 1 second between every iteration"
        sleep 1
done
```

$# Shell Variable

echo $# outputs the number of positional parameters of your script.

```
if [[ $# -ne 1 ]]; then
        echo “Please use only one argument for file name”
        exit 1
fi
```

while loop example:

```
#!/bin/bash

while [ "$#"  -ne 1 ]; do
        #your commands here
done
```

until

until

```
until CONDITION;
        do
    CONSEQUENT-COMMANDS;
done
```

```
#!/bin/bash
counter=0
until [ $counter –ge 5 ]
do
        echo Counter: $counter
        ((counter++))
done
```

- Very similar to **while** loop
  - except that the loop executes until the **TEST-COMMAND** executes successfully